

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

最短路三大算法

Shortest Path: Floyd, Dijkstra and SPFA

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 29 日

目录

1. 最短路问题概述
2. Floyd 算法
3. Dijkstra 算法
4. SPFA 算法
5. 总结



目录

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

1. 最短路问题概述
2. Floyd 算法
3. Dijkstra 算法
4. SPFA 算法
5. 总结

什么是最短路

最短路问题：给定一个带权图，每条边有一个权值，求从某个点出发到另一个点的“代价最小”的路径。

两类常见问题

- **单源最短路**：给定源点 s，求 s 到所有其他点的最短距离
- **多源最短路**：求任意两点之间的最短距离

本课覆盖的三种算法

- Floyd：多源最短路，代码最短，适合点数不大的图
- Dijkstra：单源最短路，非负权图的首选
- SPFA：单源最短路，可处理负权，还能检测负环

同样求最短路，三种算法各有适用的场景

三种算法对比

算法	问题类型	适用边权	时间复杂度	能否检测负环
Floyd	多源	任意（无负环）	$O(n^3)$	否
Dijkstra	单源	非负	$O(m \log n)$	否
SPFA	单源	任意	最坏 $O(nm)$	可以

选型建议

- 点数小、要任意两点间的距离：Floyd
- 边权非负、单源：Dijkstra
- 有负权边，或需要检测负环：SPFA

先看边权是否非负，再决定用哪个算法

目录

1 最短路问题概述

2 Floyd 算法

3 Dijkstra 算法

4 SPFA 算法

5 总结



例题：多源最短路

题目：给定 n 个点、m 条带权有向边的图，1 号点可以到达所有点。求任意两点 u 到 v 的最短距离。若 u 无法到达 v，输出 -1。

示例

- $n = 4$，**边** $(1, 2, 3)$、$(1, 3, 8)$、$(2, 3, 2)$、$(2, 4, 5)$、$(3, 4, 1)$
- 1 **到** 4：**路径** $1 \to 2 \to 3 \to 4$，**距离** $3 + 2 + 1 = 6$
- 2 **到** 4：**路径** $2 \to 3 \to 4$，**距离** $2 + 1 = 3$
- 3 **到** 4：**路径** $3 \to 4$，**距离** 1

Floyd：先看原始的三维 DP

状态定义：$f[k][i][j]$ 表示“只允许经过 $1, 2, \ldots, k$ 号点作为中转点时，i 到 j 的最短距离”。

边界：$k = 0$

- 不允许任何中转点，只能走一条直接边
- $f[0][i][j]$ 就是 i 到 j 的边权（无边则是 INF）

转移：考虑第 k 号点用不用作中转

- 不用 k 中转：$f[k-1][i][j]$
- 用 k 中转：$f[k-1][i][k] + f[k-1][k][j]$

$$f[k][i][j] = \min\left(f[k-1][i][j],\ f[k-1][i][k] + f[k-1][k][j]\right)$$

最终答案：$f[n][i][j]$，中转点放开到全部 n 个点。

Floyd：为什么可以从三维降到二维

关键观察：$f[k][\cdot][\cdot]$ 只依赖上一层 $f[k-1][\cdot][\cdot]$。

于是可以原地滚动更新

- 把第一维 k 去掉，用二维数组 $d[i][j]$ 原地覆盖
- 第 k 轮，用当前 d 里的值直接算新的 d

降维后仍然正确的关键

- 本轮要用 $d[i][k]$ 与 $d[k][j]$ 两个值
- 若 $d[i][k]$ 变小，路径会绕回 k，无负环时不会更优
- 所以两者本轮内保持稳定，可以安全使用

无负环保证 $d[i][k]$ 稳定，才允许降维

Floyd：为什么 k 必须放最外层

降维依赖“第 k 轮只依赖第 $k-1$ 轮”的语义

正确做法

- 第 k 轮：允许 $1..k$ 号点作为中转
- 算完这一轮，$1..k$ 的中转就已经被充分利用
- 下一轮再放开 $k+1$，依赖的是上一轮完整结果

错误做法

- 把 i 或 j 放在最外层，k 放在最内层
- 中转点没有被“层层放开”，结果会错

直观类比

- 类似逐层加料：先加第一种调料，再加第二种，不能打乱顺序

k 在外层，是 Floyd 正确性的前提

Floyd：三件关键初始化

第一步：对角线上填 0

- $d[i][i] = 0$，自己到自己距离为 0

第二步：直接边填边权

- 对每条边 (u, v, w)，设 $d[u][v] = w$
- 若同一对点有多条边，取最小那条

第三步：其余位置填 INF

- $d[i][j] = \text{INF}$，表示目前不知道 i 到 j 的路径
- 加法时要注意“INF 加任何数还是 INF”，避免溢出

对角线 0、直接边填边权、其余 INF

Floyd：走一遍示例

初始矩阵（INF 记作 ∞），行是起点，列是终点。

	1	2	3	4
1	0	3	8	∞
2	∞	0	2	5
3	∞	∞	0	1
4	∞	∞	∞	0

过程要点

- $k=1$：**其他点到 1 都是 ∞，没有可改进**
- $k=2$：$d[1][3]=\min(8,3+2)=5$，$d[1][4]=8$
- $k=3$：$d[1][4]=\min(8,5+1)=6$，$d[2][4]=\min(5,3)=3$
- $k=4$：**没有更短路径，矩阵不再变化**

最终答案：1 到 4 是 6，2 到 4 是 3，3 到 4 是 1。

Floyd 代码


```cpp
const int INF = 0x3f3f3f3f;
const int MAXN = 305;
int d[MAXN][MAXN];
void init(int n) {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            d[i][j] = (i == j) ? 0 : INF;
}
void addEdge(int u, int v, int w) {
    d[u][v] = min(d[u][v], w);    // 重边取最小
}
void floyd(int n) {
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                if (d[i][k] + d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];
}
```

Floyd 代码

复杂度 $O(n^3)$，三重循环顺序：k 最外层，i 次之，j 最内层。

目录

1 最短路问题概述

2 Floyd 算法

3 Dijkstra 算法

4 SPFA 算法

5 总结

例题：单源最短路

题目：给定 n 个点、m 条带非负权有向边的图，给定源点 s。求从 s 到所有点的最短距离，若不可达输出 -1。

示例

- $n = 5$，**源点** $s = 1$
- **边**：$(1, 2, 2)$、$(1, 3, 4)$、$(2, 3, 1)$、$(2, 4, 7)$、$(3, 5, 3)$、$(4, 5, 1)$
- **从 1 出发**：**到 2 是 2**，**到 3 是 3**，**到 4 是 9**，**到 5 是 6**

Dijkstra：先理解“松弛”

松弛是三种最短路算法共同的“微观操作”，先把它讲透。

一句话定义

- 已知 s 到 u 的距离 $dis[u]$，边 (u, v) 权值为 w
- 如果 $dis[u] + w$ 比当前 $dis[v]$ 更小，就用它替换 $dis[v]$

通俗理解

- $dis[v]$ 是“从 s 到 v 目前已知的最省走法”
- 发现一条新路线：先到 u，再走一步到 v
- 如果新路线更便宜，就替换掉旧的

代码写法

```
if (dis[v] 大于 dis[u] + w) dis[v] = dis[u] + w;
```

松弛 = 用中转点 u 试探邻居 v，能更短就更新

Dijkstra：整体循环的流程图

初始化

- $dis[s] = 0$，其余 INF；源点入队，所有点未锁定

主循环：只要队列非空，就一直重复三步

- 第 1 步：从队列中取出 dis 最小的点 u
- 第 2 步：若 u 已锁定过，直接丢弃，回到循环开头
- 第 3 步：锁定 u，并用 u 松弛所有邻居

结束：队列为空，所有可达点的 dis 就是最终答案。

取最小、锁定、松弛，三步循环，直到队列空

Dijkstra：循环第一二步在做什么

第 1 步：取出 dis 最小的未锁定点 u

- 队列是按 dis 排序的最小堆
- 弹出的就是当前所有候选里最近的那个

第 2 步：若 u 已锁定过，直接丢弃

- 同一个点可能多次入队，第一次弹出时已锁定
- 重复入队 + 弹出时检查，称为**懒删除**
- 不跳过会用旧值去松弛，把邻居 dis 推大

取最小 + 懒删除：跳过旧记录

Dijkstra：循环第三步在做什么

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

第 3 步：锁定 u，用 u 松弛所有邻居

锁定 u

- 把 $vis[u]$ 置真，此后 $dis[u]$ 不再改变

松弛邻居 v

- 对每条出边 (u, v, w)，若 $dis[u] + w$ 更小就更新
- 更新成功时，把新值 $(dis[v], v)$ 压入优先队列

为什么锁定后不改 $dis[u]$：边权非负，未来只会更长。

锁定后定稿，立刻传播给邻居

Dijkstra：为什么锁定是安全的

关键问题：$dis[u]$ 已经最小，凭什么能确定它就是最终答案？

反证

- 假设 $dis[u]$ 还不是真正最短路
- 真实最短路一定经过某个还没锁定的点 x
- x 的当前 dis 不会小于 $dis[u]$（因为 u 最小）
- 从 x 再往后走边权非负，只会更长，矛盾

推理依赖两个条件

- 边权非负：否则越走越短，反证不成立
- u 是队列中最小：所以 x 不比 u 小

最小 + 非负权 = 可以锁定

Dijkstra：循环为什么一定会结束

每一轮循环都至少发生一件事

- 要么锁定一个新点（$vis[u]$ 由假变真）
- 要么丢弃一个已经锁定过的重复项

上界估算

- 最多 n 个点会被锁定
- 每次松弛成功才入队，总入队次数不超过 m
- 队列最多 $n+m$ 次弹出，有限次后结束

每轮都有进展，有限次必定结束，总时间 $O(m \log m)$

Dijkstra：两个数组与队列的分工

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

dis 数组

- $dis[v]$：目前已知的 s 到 v 的最短距离
- 初始化 $dis[s] = 0$，其余 INF；松弛时可能被改小

vis 数组

- $vis[v]$ 表示 v 是否已被锁定
- 第一次从队列中弹出时置真，防止被反复锁定

优先队列

- 装 (dis, v) 候选点对，同一点可入队多次
- 弹出时靠 vis 判断是否过期

dis 存估计，vis 标定稿，队列排候审

Dijkstra：四步法整理

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dis[v]$ 是 s 到 v 的当前估计，$vis[v]$ 表示是否已锁定
- 转移：取出 dis 最小的未锁定点 u，用 $dis[u]+w$ 松弛邻居
- 边界：$dis[s]=0$，其余 INF；队列初始只放 s
- 顺序：循环“取最小、锁定、松弛”，直到队列为空

两个常见错误

- 忘记跳过已锁定的点，同一轮内把 dis 反复改小
- 松弛后没有把新值入队，候选点被漏掉

一轮锁一个点，锁定后立刻松弛邻居

Dijkstra：走一遍示例

初始：$dis[1]=0$，**其余** ∞，**所有点未锁定。队列** $\{(0,1)\}$。

轮次	锁定点	松弛后 dis 的变化
1	1 (0)	$dis[2]=2$，$dis[3]=4$；2、3 入队
2	2 (2)	$dis[3]=3$，$dis[4]=9$；3、4 入队
3	3 (3)	$dis[5]=6$；5 入队
4	5 (6)	无出边，无更新
5	4 (9)	$dis[5]$ 已为 6，不再更新

最终 dis：0, 2, 3, 9, 6。

观察

- 第 4 轮锁定的是点 5 而非点 4，因为 $dis[5]=6$ 比 $dis[4]=9$ 小
- 队列弹出顺序完全由 dis 决定，而不是点的编号

Dijkstra：朴素版 vs 优先队列版

朴素版

- 每轮扫描全部未锁定点，找 dis 最小的那个
- 每轮 $O(n)$，总复杂度 $O(n^2)$，适合稠密图

优先队列版

- 最小堆维护候选 dis，弹出即当前最近点
- 松弛成功才入堆，弹出时靠 vis 跳过过期项（懒删除）
- 总复杂度 $O(m \log m)$，适合稀疏图

稠密图用朴素版，稀疏图用优先队列版

Dijkstra 代码：优先队列版


```cpp
typedef pair<int, int> pii;
vector<pii> g[MAXN];          // 邻接表，存 (to, weight)
int dis[MAXN];
bool vis[MAXN];
void dijkstra(int s, int n) {
    fill(dis, dis + n + 1, INF);
    dis[s] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        int u = pq.top().second;
        pq.pop();
        if (vis[u]) continue;     // 懒删除：已锁定的点跳过
        vis[u] = true;            // 锁定 u
        for (auto& e : g[u]) {
            int v = e.first, w = e.second;
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;   // 松弛成功
```

Dijkstra 代码：优先队列版


```
19                  pq.push({dis[v], v});  // 新值入队
20              }
21          }
22      }
23  }
```

目录

1 最短路问题概述

2 Floyd 算法

3 Dijkstra 算法

4 SPFA 算法

5 总结



例题：含负权的单源最短路

题目：给定 n 个点、m 条带权有向边（权值可以为负）的图，给定源点 s。求 s 到所有点的最短距离。若存在从 s 可达的负环，输出“存在负环”。

示例

- $n = 4$，**源点** $s = 1$
- **边**：$(1, 2, 3)$、$(1, 3, 5)$、$(2, 3, -2)$、$(3, 4, 1)$
- **从 1 出发**：**到 2 是 3**，**到 3 是 1**，**到 4 是 2**
- **若再加边** $(3, 2, -1)$，**则存在负环，应报告存在负环**

SPFA：核心思路

出发点：Dijkstra 因“已锁定点不能改”而在负权下失效；SPFA 不锁定，谁被更新谁重新参与。

三个关键角色

- **队列**：存放“dis 刚刚变小、有待扩散”的点
- inq **数组**：标记某点当前是否在队列中
- **松弛**：若 $dis[u]+w$ 比 $dis[v]$ 更小，就更新 $dis[v]$

和 BFS 的区别：BFS 每点只入队一次，SPFA 可入队多次。

被更新的点入队，重新参与松弛

SPFA：整体循环的流程图

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

初始化

- $dis[s] = 0$，其余 INF；源点入队，$inq[s]$ 真，$cnt[s] = 1$

主循环：只要队列非空，就一直重复三步

- 第 1 步：弹出队首 u，把 $inq[u]$ 置假
- 第 2 步：遍历 u 每条出边，用 $dis[u] + w$ 松弛邻居 v
- 第 3 步：若 $dis[v]$ 变小且 $inq[v]$ 为假，把 v 入队

结束：队列空，没有任何点还能被更新，dis 就是最终答案。

弹出、松弛、按需入队，三步循环，直到队列空

SPFA：循环每一步在做什么

第 1 步：弹出队首 u，把 $inq[u]$ 置假

- $inq[u]$ 置假，表示 u 已暂时离开队列
- 若后续 $dis[u]$ 又被改小，可以再次入队

第 2 步：用 u 松弛所有邻居

- 对每条出边 (u, v, w)，若 $dis[u] + w$ 更小就更新

第 3 步：新值变小且 v 不在队列时入队

- 入队后置 $inq[v]$ 真、$cnt[v]$ 自增
- 若 v 已在队列中，不重复入队，等它弹出时再用新值

先出队，再松弛，能变小才重新入队

SPFA：为什么负环要单独处理

没有负环时

- 每次变小都对应一条更短路径，长度有下界，更新有限
- 队列最终会空，算法自然结束

有可达负环时

- 绕负环一圈路径更短，dis 会一直变小
- 队列永不空，程序死循环，必须主动判定

判定依据

- n 个点的最短路最多 $n-1$ 条边
- 若某点入队 n 次，最短路已含 n 条边，必有负环

没有负环一定停，有负环靠 cnt 拦下

SPFA：负环检测的实现

核心变量：$cnt[v]$ 记录点 v 的入队次数，$cnt[s]$ 初值为 1。

何时自增

- 只有当 v 真正入队时，才执行 $cnt[v]$ 加 1
- 若 v 已在队列中，本轮不入队，也就不自增

何时报告负环

- 入队后立刻检查：若 $cnt[v] \geq n$，直接返回“存在负环”

两个常见错误

- 每次松弛成功都 cnt 加 1（错）：同一轮多次变小会误判
- 忘记初始化 $cnt[s] = 1$（错）：源点应算第 1 次入队

入队时才自增，达到 n 就判定负环

SPFA：四步法整理

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dis[v]$ 是 s 到 v 的最短估计，$inq[v]$ 表示是否在队列中
- 转移：弹队首 u，对每条 (u, v, w)，若 $dis[u] + w$ 更小就更新
- 边界：$dis[s] = 0$，其余 INF；队列初始只有 s，$inq[s]$ 与 $cnt[s]$ 置好
- 顺序：循环到队列空；每个被改小的邻居若不在队列就入队

两个常见错误

- 出队时忘记把 $inq[u]$ 置假，导致 u 永远不再入队
- 入队前没检查 $inq[v]$，同一轮反复入队，可能误判负环

出队要清标记，入队要判重复

SPFA：走一遍示例

初始：$dis[1]=0$，**其余** ∞。**队列** $\{1\}$，$inq[1]$ **为真**。

步骤	弹出的点	松弛结果
1	1 $(dis=0)$	$dis[2]=3$，$dis[3]=5$，2、3 入队
2	2 $(dis=3)$	$dis[3]=\min(5,1)=1$，3 已在队，不重复入队
3	3 $(dis=1)$	$dis[4]=2$，4 入队
4	4 $(dis=2)$	无出边，队列空，结束

最终 dis：0，3，1，2。

观察

- 点 3 被更新两次：先 5 后 1
- 因为 3 第一次入队后还没出队，第二次变小时不重复入队

SPFA 代码：含负环检测


```cpp
int dis[MAXN];
bool inq[MAXN];
int cnt[MAXN];            // 入队次数，用于检测负环
bool spfa(int s, int n) {
    fill(dis, dis + n + 1, INF);
    fill(inq, inq + n + 1, false);
    fill(cnt, cnt + n + 1, 0);
    dis[s] = 0;
    queue<int> q;
    q.push(s);
    inq[s] = true;
    cnt[s] = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for (auto& e : g[u]) {
            int v = e.first, w = e.second;
```

SPFA 代码：含负环检测

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19              if (dis[v] > dis[u] + w) {
20                  dis[v] = dis[u] + w;
21                  if (!inq[v]) {
22                      q.push(v);
23                      inq[v] = true;
24                      cnt[v]++;
25                      if (cnt[v] >= n) return false;     // 存在负环
26                  }
27              }
28          }
29      }
30      return true;
31  }
```

SPFA：为什么它是最坏 $O(nm)$

最坏情况

- 在最坏图上，每个点可能被更新 $O(n)$ 次
- 每次更新都要遍历该点所有出边，总工作量最坏 $O(nm)$

常见误解

- 网上流传的“SPFA 均摊 $O(m)$”，只是实验观察
- 出题人可以构造卡 SPFA 的图，让它跑到 $O(nm)$ 附近

什么时候用 SPFA

- 需要处理负权边，或需要检测负环
- 数据规模不大，或者图是随机生成的稀疏图

有负权才上 SPFA，非负权请老老实实用 Dijkstra

目录

1 最短路问题概述

2 Floyd 算法

3 Dijkstra 算法

4 SPFA 算法

5 总结



总结（一）：三种算法核心思想

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

- Floyd：枚举中转点，动态规划，$O(n^3)$ 处理多源
- Dijkstra：贪心选最小，逐步锁定，$O(m \log n)$ 处理非负单源
- SPFA：队列驱动，反复松弛，最坏 $O(nm)$，能检测负环

共同要素

- 都需要一个 dis 数组维护“当前最优估计”
- 都在做“松弛”操作：尝试用更短的路径替换当前估计
- 边界都是 $dis[s] = 0$，其余 INF

松弛是核心操作，区别在于“按什么顺序松弛”

总结（二）：选型决策树

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

从问题出发一步步选

- 要任意两点间的距离：直接用 Floyd
- 只要从固定源点出发：
  - 边权都非负：用 Dijkstra（优先队列版）
  - 有负权边：用 SPFA，顺带检查负环
- 需要判断负环：用 SPFA 的入队次数，或 Bellman-Ford

易错点

- Floyd 忘记 k 放在最外层
- Dijkstra 用在负权图上
- SPFA 忘记清空 inq 和 cnt 数组

选对算法，写对顺序，别忘清空状态

总结（三）：下一步学什么

进阶主题

- 最短路计数：同时维护“最短距离”和“方案数”
- 最短路树：从源点出发，把每个点的最优前驱连起来
- 次短路：允许路径“略差一点点”的问题
- 分层图最短路：把状态（如“用了多少次机会”）也编入图中

相关算法

- Bellman-Ford：SPFA 的“非队列版”原型
- Johnson 算法：全源最短路，含负权时用

先吃透这三种，再往外拓展

附录：Floyd 三维 DP 的完整推导

三维状态

- $f[k][i][j]$：只允许 $1..k$ 号点作中转，i 到 j 的最短路
- $k = 0$：无中转，$f[0][i][j]$ 就是直接边权

转移为什么是两种选择

- 考虑第 k 号点：要么不用它中转，要么用它中转
- 不用：$f[k-1][i][j]$；用：$f[k-1][i][k] + f[k-1][k][j]$

降维的正确性

- 若在第 k 轮更新 $d[i][k]$，说明路径经过 k 又回到 k
- 无负环时这种回路代价非负，不会更优

三维定义 + 无负环，才允许降维

附录：Dijkstra 为什么不能处理负权

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

反例

- 三个点：$1 \to 2$ 权值 1，$2 \to 3$ 权值 -3，$1 \to 3$ 权值 5
- 从 1 出发：初始 $dis[1] = 0$，$dis[2] = 1$，$dis[3] = 5$
- 第一步，选 dis 最小的未锁定点 2（$dis = 1$），锁定
- 第二步，用 2 松弛 3：$dis[3] = 1 + (-3) = -2$
- 第三步，选 3 锁定：最终 $dis[3] = -2$

问题出在哪

- 假设某个点先被锁定，但后续发现经过负权边可以更短
- “已锁定”就被破坏了，Dijkstra 的贪心不成立

非负权是 Dijkstra 的前提，越界即失效

附录：三种算法的常见应用

Floyd

- 求传递闭包（可达性）
- 求最小环（枚举 k 时顺带检查）

Dijkstra

- 地图导航：从一处到其他处的最短路径
- 网络路由：链路代价非负的场景

SPFA

- 差分约束系统：转成带负权的图求最短路
- 判负环：负环意味着系统无解

同一个算法，能在不同问题里扮演不同角色

附录：三种算法的代码骨架对照

```cpp
// Floyd：三重循环，k 在最外层
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

// Dijkstra：优先队列 + 懒删除
while (!pq.empty()) {
    int u = pq.top().second; pq.pop();
    if (vis[u]) continue;
    vis[u] = true;
    for (auto& e : g[u])
        if (dis[e.first] > dis[u] + e.second) {
            dis[e.first] = dis[u] + e.second;
            pq.push({dis[e.first], e.first});
        }
}
```

附录：三种算法的代码骨架对照

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// SPFA：队列 + inq 标记 + cnt 计数
while (!q.empty()) {
    int u = q.front(); q.pop();
    inq[u] = false;
    for (auto& e : g[u]) {
        int v = e.first, w = e.second;
        if (dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            if (!inq[v]) {
                q.push(v); inq[v] = true; cnt[v]++;
                if (cnt[v] >= n) return false;
            }
        }
    }
}
```